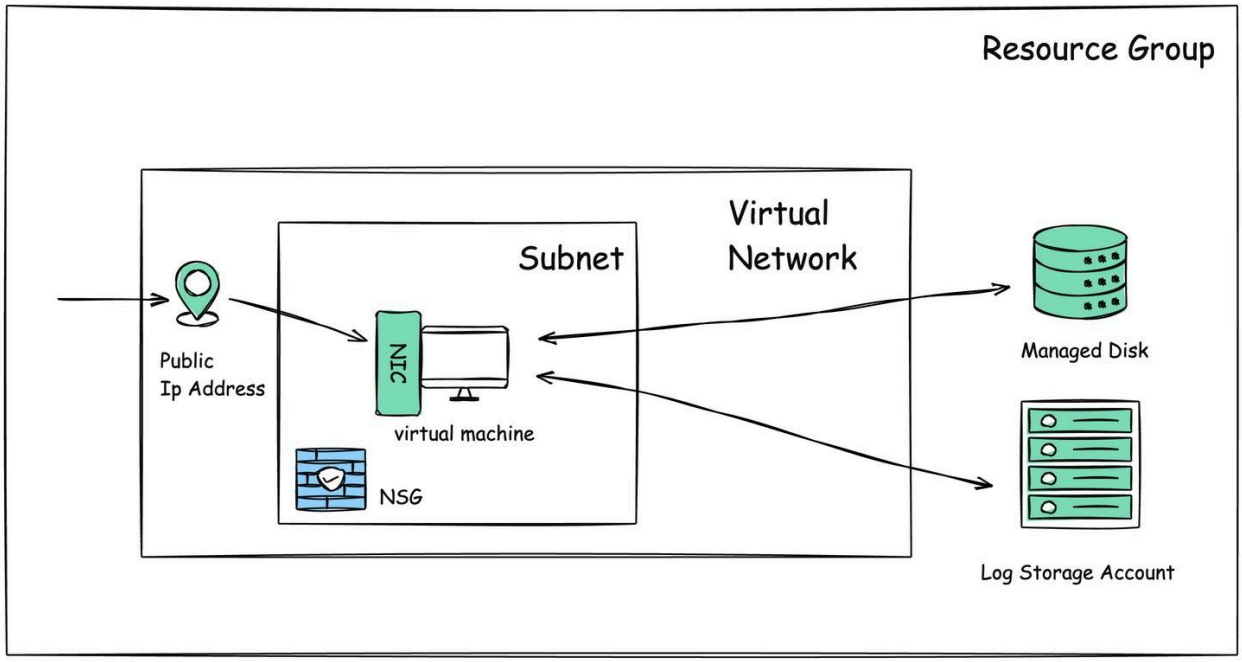
Lab: Provision an Azure Linux Virtual Machine in a Custom Virtual Network using Terraform

In this lab, you will use **Terraform** to provision an **Azure Linux Virtual Machine (VM)** inside a custom **Virtual Network (VNet)**. Along with the VM, you will define and configure all required components: subnets, a Network Security Group (NSG), a Network Interface Card (NIC), a Public IP, and a Storage Account for diagnostics.



By the end of this lab, you will have a working Linux VM deployed in Azure and accessible via SSH.

Learning Objectives

You will learn how to:

1. Write and execute Terraform scripts for Azure infrastructure.
2. Create and configure an **Azure Resource Group**.

3. Define and deploy an **Azure Virtual Network (VNet)** with a subnet.
4. Create and associate a **Network Security Group (NSG)** to control traffic.
5. Provision a **Public IP address**.
6. Deploy a **Network Interface Card (NIC)** and attach it to the VM.
7. Create a **Storage Account** for diagnostics.
8. Deploy a **Linux VM** with SSH access.
9. Use Terraform commands to provision and destroy infrastructure.

Deployment Flow

1. Write Terraform scripts to define Azure resources.
2. Initialize Terraform and validate the configuration.
3. Apply the configuration to deploy resources in Azure.
4. Connect to the VM via SSH.
5. Destroy the resources after testing.

Prerequisites

Before starting, ensure you have:

- A valid **Azure subscription**.
- **Terraform** installed locally.
- **Azure CLI** installed and authenticated (az login).

- A code editor such as **VS Code**.

Steps to Follow

Step 1: Project Setup and Authentication

1. Create a new folder named **virtual_machine** in your workspace.

Authenticate to Azure:

```
az login
```

If you have multiple subscriptions, set the one you want to use:

```
 az account set --subscription "<your-subscription-id>"
```

Step 2: Define the Provider

Inside the virtual_machine folder, create a file named **providers.tf** and add:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "4.26.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your-subscription-id>"
}
```

Run:

```
terraform init
```

This initializes the project, downloads provider plugins, and creates a .terraform.lock.hcl file.

```
saurabhdhingra@Saurabhs-MacBook-Pro lab1_resource_group % cd ../lab2_virtual_machine
saurabhdhingra@Saurabhs-MacBook-Pro lab2_virtual_machine % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "4.26.0"...
- Installing hashicorp/azurerm v4.26.0...
- Installed hashicorp/azurerm v4.26.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
saurabhdhingra@Saurabhs-MacBook-Pro lab2_virtual_machine %
```

Step 3: Define Variables

Create a file named **variables.tf** and add all the required variables (for resource group, location, VNet, subnet, NIC, NSG, VM, etc.). Example:

```
variable "resource_group_name" {
 type = string
}

variable "location" {
 type = string
}

variable "tags" {
```

```
 type = map(any)
}

variable "vnet_name" {
 type = string
}

variable "cidr_block" {

 type    = list(string)
 default = ["10.0.0.0/16"]

}

variable "subnet_cidr_block" {

 type    = list(string)
 default = ["10.0.2.0/24"]

}

variable "subnet_name" {
 type = string
}

variable "public_ip_address" {
 type = string
}

variable "virtual_machine_name" {
 type = string
}

variable "disksize" {
 type    = string
 default = "Standard_DS1_v2"
}
```

```
variable "nic_name" {
 type = string
}

variable "nsg_name" {
 type = string
}

variable "ip_allocation_method" {

 type    = string
 default = "Static"

}

variable "ip_configuration_name" {

 type = string

}

variable "account_replication_type" {

 type = string

 default = "LRS"

}

variable "account_tier" {

 type = string

 default = "Standard"

}

variable "os_disk_name" {
```

```
 type = string
 }

variable "os_disk_caching" {

 type = string
 default = "ReadWrite"
 }

variable "os_disk_storage_account_type" {

 type = string
 default = "Premium_LRS"
}

variable "source_image_reference_publisher" {

 type = string

 default = "Canonical"
 }

variable "source_image_reference_offer" {

 type = string

 default = "UbuntuServer"
 }

variable "source_image_reference_sku" {

 type = string

 default = "18.04-LTS"
 }

variable "source_image_reference_version" {
```

```
 type = string

 default =  "latest"
 }

variable "vm_admin_username" {

 type = string

 default = "azureuser"
 }

variable "vm_disable_password_authentication" {

 type = bool

 default = true
 }
```

Step 4: Provide Variable Values

Create a file named **terraform.tfvars** and specify values:

```
resource_group_name   = "devops-rg"
location              = "West Europe"
tags                  = { "env" : "production" }
vnet_name             = "devops-vnet"
cidr_block            = ["10.0.0.0/16"]
subnet_cidr_block     = ["10.0.2.0/24"]
subnet_name           = "devops-subnet"
public_ip_address     = "devops-public-ip"
disksize              = "Standard_D2s_v3"
nic_name              = "devops-public-ip"
nsg_name              = "devops-nsg"
virtual_machine_name  = "devops-virtual-machine"
```

```
ip_configuration_name = "devops-ipconfiguration"
os_disk_name = "devops-os-disk"
```

Update values to suit your requirements.

Step 5: Define Azure Resources

Create separate Terraform files for each resource:

- **resource_group.tf** – defines the Resource Group.

```
resource "azurerm_resource_group" "resource_group" {
 name     = var.resource_group_name
 location = var.location

 tags = var.tags
}
```

- **vnet.tf** – defines the Virtual Network.

```
resource "azurerm_virtual_network" "virtual_network" {
 name                = var.vnet_name
 address_space       = var.cidr_block
 location            = var.location
 resource_group_name = azurerm_resource_group.resource_group.name

 tags = var.tags
}
```

- **subnet.tf** – defines the subnet.

```
resource "azurerm_subnet" "subnet" {
 name           = var.subnet_name
 resource_group_name  = azurerm_resource_group.resource_group.name
 virtual_network_name = azurerm_virtual_network.virtual_network.name
```

```
 address_prefixes     = var.subnet_cidr_block
}
```

- **nsg.tf** – defines the Network Security Group with SSH (22) and HTTP (80) inbound rules.

```
resource "azurerm_network_security_group" "network_security_group" {
 name                = var.nsg_name
 location            = var.location
 resource_group_name = azurerm_resource_group.resource_group.name

 security_rule {
   name                       = "allow_ssh"
   priority                   = 1001
   direction                  = "Inbound"
   access                     = "Allow"
   protocol                   = "Tcp"
   source_port_range          = "*"
   destination_port_range     = "22"
   source_address_prefix      = "*"
   destination_address_prefix = "*"
 }

 security_rule {
   name                       = "allow_http"
   priority                   = 1002
   direction                  = "Inbound"
   access                     = "Allow"
   protocol                   = "Tcp"
   source_port_range          = "*"
   destination_port_range     = "80"
   source_address_prefix      = "*"
   destination_address_prefix = "*"
 }
 tags = var.tags
}
```

- **public_ip_address.tf** – provisions a Public IP.

```
resource "azurerm_public_ip" "public_ip" {
 name                = var.public_ip_address
 location            = var.location
 resource_group_name = azurerm_resource_group.resource_group.name
 allocation_method   = var.ip_allocation_method

 tags = var.tags
}
```

- **nic.tf** – provisions a NIC and associates it with the NSG.

```
resource "azurerm_network_interface" "network_interface_card" {
 name                = var.nic_name
 location            = var.location
 resource_group_name = azurerm_resource_group.resource_group.name

 ip_configuration {
  name                          = var.ip_configuration_name
  subnet_id                     = azurerm_subnet.subnet.id
  private_ip_address_allocation = "Dynamic"
  public_ip_address_id          = azurerm_public_ip.public_ip.id
 }

 tags = var.tags
}

# Connect the security group to the network interface
resource "azurerm_network_interface_security_group_association" "example" {
 network_interface_id      = azurerm_network_interface.network_interface_card.id
 network_security_group_id =
azurerm_network_security_group.network_security_group.id
}
```

- **storage_account.tf** – provisions a Storage Account for diagnostics (uses the random provider for unique names).

```
resource "random_id" "random_id" {
 keepers = {
  # Generate a new ID only when a new resource group is defined
  resource_group = azurerm_resource_group.resource_group.name
 }

 byte_length = 8
}

resource "azurerm_storage_account" "mystorageaccount" {
 name                     = "diag${random_id.random_id.hex}"
 resource_group_name      = azurerm_resource_group.resource_group.name
 location                 = var.location
 account_replication_type = var.account_replication_type
 account_tier             = var.account_tier

 tags = var.tags
}
```

- **secure_tls_key.tf** – uses the tls provider to generate SSH keys.

```
resource "tls_private_key" "ssh_key" {
 algorithm = "RSA"
 rsa_bits  = 4096
}
output "tls_private_key" {
 value     = tls_private_key.ssh_key.private_key_pem
 sensitive = true
}
```

- **virtual_machine.tf** – defines the Linux VM with OS disk settings, image reference (Ubuntu), admin user, and SSH key.

```
resource "azurerm_linux_virtual_machine" "virtual_machine" {
 name                = var.virtual_machine_name
 location            = var.location
 resource_group_name   = azurerm_resource_group.resource_group.name
 network_interface_ids =
[azurerm_network_interface.network_interface_card.id]
 size                = var.disksize

 os_disk {
   name                 = var.os_disk_name
   caching              = var.os_disk_caching
   storage_account_type = var.os_disk_storage_account_type
 }

 source_image_reference {
   publisher = var.source_image_reference_publisher
   offer     = var.source_image_reference_offer
   sku       = var.source_image_reference_sku
   version   = var.source_image_reference_version
 }

 computer_name                   = var.virtual_machine_name
 admin_username                  = var.vm_admin_username
 disable_password_authentication =
var.vm_disable_password_authentication

 admin_ssh_key {
   username   = var.vm_admin_username
   public_key = tls_private_key.ssh_key.public_key_openssh
 }

 boot_diagnostics {
   storage_account_uri =
azurerm_storage_account.mystorageaccount.primary_blob_endpoint
 }

 tags = var.tags
}
```

- **output.tf** – outputs the VM's public IP address after deployment.

```
output "public_ip_address" {

  value =
azurerm_linux_virtual_machine.virtual_machine.public_ip_address
}
```

Update the providers.tf with the following code to include the random_id and tls

```
terraform {
 required_providers {
   azurerm = {
     source  = "hashicorp/azurerm"
     version = "4.26.0"
   }

   random = {
     source  = "hashicorp/random"
     version = "3.7.1"
   }

   tls = {
     source  = "hashicorp/tls"
     version = "4.0.6"
   }
 }
}

provider "azurerm" {
 # Configuration options

 features {

 }
```

```
 subscription_id = "<your-subscription-id>"
}

provider "random" {
}

provider "tls" {

}
```

Make sure to update the subscription id or just copy the new providers and update the configuration

Re-run:

```
terraform init
```

Step 6: Deploy the Infrastructure

Format configuration:

```
terraform fmt
```

Validate configuration:

```
terraform validate
```

Preview the deployment:

```
terraform plan
```

```
  # tls_private_key.ssh_key will be created
  + resource "tls_private_key" "ssh_key" {
      + algorithm                     = "RSA"
      + ecdsa_curve                   = "P224"
      + id                            = (known after apply)
      + private_key_openssh           = (sensitive value)
      + private_key_pem               = (sensitive value)
      + private_key_pem_pkcs8         = (sensitive value)
      + public_key_fingerprint_md5    = (known after apply)
      + public_key_fingerprint_sha256 = (known after apply)
      + public_key_openssh            = (known after apply)
      + public_key_pem                = (known after apply)
      + rsa_bits                      = 4096
    }

Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + public_ip_address = (known after apply)
  + tls_private_key   = (sensitive value)

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
```

Apply the configuration:

```
terraform apply --auto-approve
```

```
azurerm_storage_account.mystorageaccount: Still creating... [50s elapsed]
azurerm_storage_account.mystorageaccount: Still creating... [1m0s elapsed]
azurerm_storage_account.mystorageaccount: Still creating... [1m10s elapsed]
azurerm_storage_account.mystorageaccount: Creation complete after 1m17s [id=/subscriptions/50cff209-0e41-4560-a131-5c873e
c03be0/resourceGroups/devops-rg/providers/Microsoft.Storage/storageAccounts/diagf4caaeed034e03db]
azurerm_linux_virtual_machine.virtual_machine: Creating...
azurerm_linux_virtual_machine.virtual_machine: Still creating... [10s elapsed]
azurerm_linux_virtual_machine.virtual_machine: Still creating... [20s elapsed]
azurerm_linux_virtual_machine.virtual_machine: Still creating... [30s elapsed]
azurerm_linux_virtual_machine.virtual_machine: Still creating... [40s elapsed]
azurerm_linux_virtual_machine.virtual_machine: Still creating... [50s elapsed]
azurerm_linux_virtual_machine.virtual_machine: Creation complete after 55s [id=/subscriptions/50cff209-0e41-4560-a131-5c8
73ec03be0/resourceGroups/devops-rg/providers/Microsoft.Compute/virtualMachines/devops-virtual-machine]

Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

public_ip_address = "108.142.124.139"
tls_private_key = <sensitive>
```

Step 7: Connect to the VM

Save the generated private key:

```
terraform output tls_private_key >> key.pem
```

```
chmod 400 key.pem
```

SSH into the VM (replace with your VM's public IP):

```
ssh -i ./key.pem azureuser@<public-ip>
```

If successful, you are connected to the Linux VM provisioned by Terraform.

Step 8: Clean Up Resources

To delete everything created by this lab:

```
terraform destroy --auto-approve
```

Conclusion

In this lab, you:

- Defined and provisioned a complete Azure environment using Terraform.
- Built a Virtual Network, subnet, NSG, NIC, Public IP, Storage Account, and Linux VM.
- Connected to the VM via SSH.
- Learned how to manage the full lifecycle of resources using Terraform commands.